

Binary Classification of Obesity Risk Using Behavioral and Physical Attributes

Project Context

This project focuses on building an end-to-end predictive machine learning pipeline using structured health-related data.

The goal is to demonstrate proficiency in problem framing, model comparison, scaling, hyperparameter optimization, and model interpretation, aligned with applied analytics and Business Intelligence practices.

The dataset used is the UCI Estimation of Obesity Levels Based on Eating Habits and Physical Condition, which contains demographic, behavioral, and physical attributes for individuals, along with labeled obesity levels.

[UCI Repository](#)

Problem Definition

Binary Classification Task

We reformulate the original multi-class obesity labels into a binary, decision-oriented target:

0 = No Obesity (Normal Weight + Insufficient Weight)

1 = Obesity Risk (Overweight + All Obesity categories)

This framing transforms the problem from academic categorization to a practical risk-identification task, suitable for real-world screening, intervention prioritization, or policy analysis.

```
In [1]: import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report
```

```
In [2]: #pip install ucimlrepo
```

```
In [3]: from ucimlrepo import fetch_ucirepo
```

```
# fetch dataset
estimation_of_obesity_levels_based_on_eating_habits_and_physical_condition = fetch_

# data (as pandas dataframes)
X = estimation_of_obesity_levels_based_on_eating_habits_and_physical_condition.data
y = estimation_of_obesity_levels_based_on_eating_habits_and_physical_condition.data
```

```
# metadata print(estimation_of_obesity_levels_based_on_eating_habits_and_physical_condition.metadata)
```

```
In [4]: # variable information
print(estimation_of_obesity_levels_based_on_eating_habits_and_physical_condition.va
```

	name	role	type	demographic \
0	Gender	Feature	Categorical	Gender
1	Age	Feature	Continuous	Age
2	Height	Feature	Continuous	None
3	Weight	Feature	Continuous	None
4	family_history_with_overweight	Feature	Binary	None
5	FAVC	Feature	Binary	None
6	FCVC	Feature	Integer	None
7	NCP	Feature	Continuous	None
8	CAEC	Feature	Categorical	None
9	SMOKE	Feature	Binary	None
10	CH20	Feature	Continuous	None
11	SCC	Feature	Binary	None
12	FAF	Feature	Continuous	None
13	TUE	Feature	Integer	None
14	CALC	Feature	Categorical	None
15	MTRANS	Feature	Categorical	None
16	NObeyesdad	Target	Categorical	None

		description	units	missing_values	
0			None	None	no
1			None	None	no
2			None	None	no
3			None	None	no
4	Has a family member suffered or suffers from o...		None		no
5	Do you eat high caloric food frequently?		None		no
6	Do you usually eat vegetables in your meals?		None		no
7	How many main meals do you have daily?		None		no
8	Do you eat any food between meals?		None		no
9	Do you smoke?		None		no
10	How much water do you drink daily?		None		no
11	Do you monitor the calories you eat daily?		None		no
12	How often do you have physical activity?		None		no
13	How much time do you use technological devices...		None		no
14	How often do you drink alcohol?		None		no
15	Which transportation do you usually use?		None		no
16	Obesity level		None		no

```
X.head()
```

```
In [5]: X.shape, y.shape
```

```
Out[5]: ((2111, 16), (2111, 1))
```

The original multiclass obesity label is converted into a binary target. Normal and underweight individuals are labeled as no risk (0), while all overweight and obesity

categories are labeled as risk (1).

```
In [6]: y.loc[:, "obesity_risk"] = y["NObeyesdad"].apply(
        lambda x: 0 if x in ["Insufficient_Weight", "Normal_Weight"] else 1
    )
```

C:\Users\biauser\AppData\Local\Temp\ipykernel_8808\3532476470.py:1: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
y.loc[:, "obesity_risk"] = y["NObeyesdad"].apply(
```

```
In [7]: y["obesity_risk"].value_counts(normalize=True)
```

```
Out[7]: obesity_risk
1      0.735197
0      0.264803
Name: proportion, dtype: float64
```

The label NObeyesdad is medically defined using BMI BMI is directly computable from Weight and Height So the model had direct access to the label logic

```
In [8]: #Those two variables almost fully determine BMI
        #BMI is implicitly embedded in the obesity Label

        X_reduced = X.drop(columns=["Weight", "Height"])
```

```
In [9]: from sklearn.model_selection import train_test_split

        X_train, X_test, y_train, y_test = train_test_split(
            X_reduced,
            y["obesity_risk"],
            test_size=0.2,
            random_state=42,
            stratify=y["obesity_risk"]
        )
```

```
In [10]: numeric_features = X_reduced.select_dtypes(include=["int64", "float64"]).columns
         categorical_features = X_reduced.select_dtypes(include=["object"]).columns
```

```
In [11]: from sklearn.compose import ColumnTransformer
         from sklearn.preprocessing import OneHotEncoder, StandardScaler
```

```
In [12]: preprocess_no_scaling = ColumnTransformer(
        transformers=[
            ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"), categorical_f
            ("num", "passthrough", numeric_features)
        ]
    )

    preprocess_standard = ColumnTransformer(
        transformers=[
```

```

        ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"), categorical_f
        ("num", StandardScaler(), numeric_features)
    ]
)

```

```

In [13]: from sklearn.pipeline import Pipeline
        from sklearn.linear_model import LogisticRegression
        from sklearn.ensemble import RandomForestClassifier
        from sklearn.metrics import roc_auc_score, accuracy_score
        import pandas as pd

```

```

In [14]: models = {
        "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
        "Random Forest": RandomForestClassifier(random_state=42)
    }

    preprocessors = {
        "No Scaling": preprocess_no_scaling,
        "StandardScaler": preprocess_standard
    }

```

```

In [15]: results = []

        for prep_name, prep in preprocessors.items():
            for model_name, model in models.items():

                pipe = Pipeline([
                    ("preprocess", prep),
                    ("model", model)
                ])

                pipe.fit(X_train, y_train)
                y_proba = pipe.predict_proba(X_test)[: , 1]
                y_pred = pipe.predict(X_test)

                results.append({
                    "Model": model_name,
                    "Scaling": prep_name,
                    "ROC-AUC": roc_auc_score(y_test, y_proba),
                    "Accuracy": accuracy_score(y_test, y_pred)
                })

```

```

C:\Users\biauser\anaconda3\envs\ds312\Lib\site-packages\sklearn\preprocessing\_encod
ers.py:261: UserWarning: Found unknown categories in columns [6] during transform. T
hese unknown categories will be encoded as all zeros
  warnings.warn(msg, UserWarning)
C:\Users\biauser\anaconda3\envs\ds312\Lib\site-packages\sklearn\preprocessing\_encod
ers.py:261: UserWarning: Found unknown categories in columns [6] during transform. T
hese unknown categories will be encoded as all zeros
  warnings.warn(msg, UserWarning)
C:\Users\biauser\anaconda3\envs\ds312\Lib\site-packages\sklearn\preprocessing\_encod
ers.py:261: UserWarning: Found unknown categories in columns [6] during transform. T
hese unknown categories will be encoded as all zeros
  warnings.warn(msg, UserWarning)
C:\Users\biauser\anaconda3\envs\ds312\Lib\site-packages\sklearn\preprocessing\_encod
ers.py:261: UserWarning: Found unknown categories in columns [6] during transform. T
hese unknown categories will be encoded as all zeros
  warnings.warn(msg, UserWarning)
C:\Users\biauser\anaconda3\envs\ds312\Lib\site-packages\sklearn\preprocessing\_encod
ers.py:261: UserWarning: Found unknown categories in columns [6] during transform. T
hese unknown categories will be encoded as all zeros
  warnings.warn(msg, UserWarning)
C:\Users\biauser\anaconda3\envs\ds312\Lib\site-packages\sklearn\preprocessing\_encod
ers.py:261: UserWarning: Found unknown categories in columns [6] during transform. T
hese unknown categories will be encoded as all zeros
  warnings.warn(msg, UserWarning)
C:\Users\biauser\anaconda3\envs\ds312\Lib\site-packages\sklearn\preprocessing\_encod
ers.py:261: UserWarning: Found unknown categories in columns [6] during transform. T
hese unknown categories will be encoded as all zeros
  warnings.warn(msg, UserWarning)

```

```
In [16]: results_df = pd.DataFrame(results)
results_df
```

```
Out[16]:
```

	Model	Scaling	ROC-AUC	Accuracy
0	Logistic Regression	No Scaling	0.921624	0.903073
1	Random Forest	No Scaling	0.978813	0.924350
2	Logistic Regression	StandardScaler	0.921681	0.900709
3	Random Forest	StandardScaler	0.978669	0.924350

Based on baseline performance, Random Forest was selected as the final model. It achieved the highest ROC-AUC and accuracy and does not require feature scaling.

```
In [17]: from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier

rf_pipeline = Pipeline([
    ("preprocess", preprocess_no_scaling),
    ("model", RandomForestClassifier(random_state=42))
])
```

```
In [18]: from sklearn.model_selection import GridSearchCV

param_grid = {
    "model__n_estimators": [100, 200],
    "model__max_depth": [None, 10, 20],
}

grid_rf = GridSearchCV(
    rf_pipeline,
    param_grid,
    scoring="roc_auc",
    cv=5
)

grid_rf.fit(X_train, y_train)
```

```
Out[18]:
```

```

    ▸ GridSearchCV ⓘ ?
      ▸ best_estimator_: Pipeline
        ▸ preprocess: ColumnTransformer ?
          ▸ cat
            ▸ OneHotEncoder ?
          ▸ num
            ▸ passthrough
        ▸ RandomForestClassifier ?

```

```
In [19]: grid_rf.best_params_
```

```
Out[19]: {'model__max_depth': None, 'model__n_estimators': 200}
```

```
In [26]: best_rf = grid_rf.best_estimator_
```

```
y_pred = best_rf.predict(X_test)
y_proba = best_rf.predict_proba(X_test)[:, 1]
```

```
C:\Users\biauser\anaconda3\envs\ds312\Lib\site-packages\sklearn\preprocessing\_encoders.py:261: UserWarning: Found unknown categories in columns [6] during transform. These unknown categories will be encoded as all zeros
  warnings.warn(msg, UserWarning)
C:\Users\biauser\anaconda3\envs\ds312\Lib\site-packages\sklearn\preprocessing\_encoders.py:261: UserWarning: Found unknown categories in columns [6] during transform. These unknown categories will be encoded as all zeros
  warnings.warn(msg, UserWarning)
```

```
In [24]: from sklearn.metrics import accuracy_score, classification_report
```

```
accuracy = accuracy_score(y_test, y_pred)
report = classification_report(y_test, y_pred)

print(f"Accuracy: {accuracy:.2f}")
```

```
print("Classification Report:")
print(report)
```

Accuracy: 0.92

Classification Report:

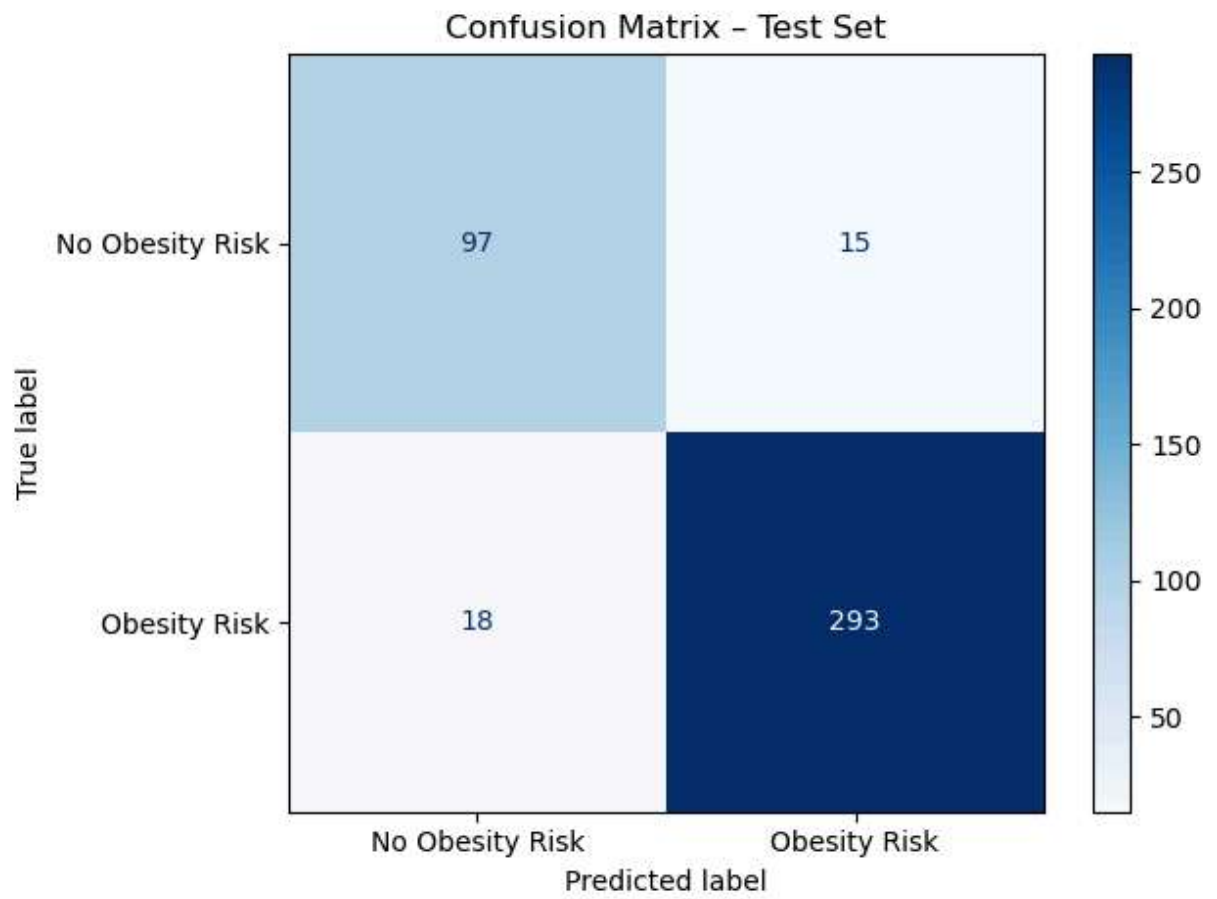
	precision	recall	f1-score	support
0	0.84	0.87	0.85	112
1	0.95	0.94	0.95	311
accuracy			0.92	423
macro avg	0.90	0.90	0.90	423
weighted avg	0.92	0.92	0.92	423

```
In [25]: from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred)

disp = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["No Obesity Risk", "Obesity Risk"]
)

disp.plot(cmap="Blues")
plt.title("Confusion Matrix - Test Set")
plt.tight_layout()
plt.show()
```



In []: